

Advanced Retrieval Augmented Generation (RAG) system for Multi-Store Retrieval

Tan Heng Joo

Singapore Institute of Technology

Singapore, 828616

2201497@sit.singaporetech.edu.sg

Abstract—Large Language Models (LLMs), while powerful, are limited by their static, pre-trained knowledge. Retrieval-Augmented Generation (RAG) addresses this by grounding LLMs in external data, but traditional frameworks that rely on a single vector store are insufficient for real-world applications involving heterogeneous data formats. This paper presents the first empirical implementation and systematic evaluation of specialized pipelines for an Agentic Multi-Store RAG (AMS-RAG) architecture, designed to intelligently query unstructured documents, structured databases, and knowledge graphs.

This paper conducts a rigorous benchmarking of vector, SQL and graph RAG pipelines, progressing from naive baselines to advanced, multi-stage configurations. Different document parsers (Nanonets OCR, SimpleDirectoryReader) were compared and the performance gains from optimisation techniques including Hypothetical Document Embeddings (HyDE), cross-encoder re-ranking, and hybrid sparse-dense retrieval were also quantified. For the SQL pipeline, query-time context enrichment techniques like Query-Time Table Retrieval (QTTR) and Query-Time Row Retrieval (QTRR) were also evaluated, while also comparing the performance of general-purpose (meta-llama/Llama-3.1-8B-Instruct) and domain-specialized (Snowflake/Arctic-Text2SQL) models. Furthermore, to address the evaluation gap for graph pipelines, a custom evaluation metric was introduced to assess the accuracy of the Text-to-Cypher generation by comparing the data returned by the generated and ground-truth queries.

This work confirms that data-specific optimisations are necessary and provides the configurations for the integrated AMS-RAG system.

Index Terms—Retrieval-Augmented Generation, RAG, Naive RAG, Advanced RAG, Modular RAG, Graph RAG, Agentic RAG, Multi-Store Retrieval

I. INTRODUCTION

Large Language Models (LLM) such as OpenAI’s GPT-4o, Google’s Gemini-2.5 Pro and xAI’s Grok-3 have demonstrated outstanding capabilities in understanding and generating human-like text as they are trained on enormous datasets that are publicly available on the internet [1]. This extensive training allows LLMs to be able to access a broad range of knowledge across diverse domains and understand the concepts from these fields simultaneously to answer the user’s question.

However, their knowledge is fundamentally limited by the static data they were trained on, which results in producing inaccurate information with confidence when asked on domains that are not well represented or not present in their training dataset. Additionally, LLMs often struggle with questions requiring real-time information or highly specialised domain

knowledge due to the “knowledge cut-off date” from static training.

To address these limitations, Retrieval-Augmented Generation (RAG) was introduced as a solution by connecting the LLMs to curated and trusted, up-to-date knowledge sources to enhance their capabilities to give up-to-date information [2]. Instead of relying solely on information from data it has been trained on (parametric memory), a RAG system first retrieves relevant information from a curated knowledge database (non-parametric memory) and uses that information to generate up-to-date responses. This enhances the prompt to the LLM responsible for generating the response, by providing up-to-date, tailored and factually grounded context in the user’s specific circumstance. While it sounds all encompassing in theory, in reality, not all data can be vectorised and stored in a vector datastore [4]. In today’s world, there are various data types besides unstructured data such as graph and structured data which can capture connections or relationships between data.

Organizations typically manage unstructured documents (PDFs, Word Documents), structured tabular data (CSV files, Excel spreadsheets) and relational databases (transaction logs, customer records), each of this format requires a specific way to be stored and retrieved. Hence, a “one-size-fits-all” approach of a single-store RAG system is significantly limited when handling this data diversity. Faced with these limitations, multi-store retrieval architectures come to mind as it can leverage the strengths of multiple types of datastores, such as graph, structured and unstructured, to tackle the various types of data to provide accurate and contextually relevant responses to the user.

II. TRADITIONAL LLMs AND THEIR LIMITATIONS

Large Language Models have demonstrated remarkable capabilities in natural language understanding and generation, achieving human-level performance across numerous tasks. However, their architecture and training methodology introduce several fundamental limitations that constrain their real-world applicability.

A. Knowledge Cut-off and Static Training Data

LLMs are trained on enormous datasets that often cover a broad range of knowledge across various domains, which allows them to absorb and utilise concepts from multiple

fields at the same time. However, these datasets are static, resulting in the inability to provide information about events or developments that occurred after their training was completed.

B. Hallucination and Factual Inaccuracy

As the LLM relies solely on static patterns captured during its training, it may produce factually incorrect or fabricated content, which is also known as hallucination. When a prompt falls outside the scope of its parametric memory, the model attempts to fill gaps in its knowledge by extracting information that is partially correlated to the prompt, resulting in responses that contain incorrect or fabricated information [3].

C. Lack of Domain-Specific Knowledge

Similar to the previous section on hallucination, LLMs cannot reference unseen content, such as information that is unique to an organisation or field. As such, it relies on its general-purpose embeddings and statistical patterns to make imprecise guesses, often producing inaccurate explanations and fabricating details.

III. BACKGROUND RELATED WORK

The integration of Large Language Models (LLMs) with external knowledge sources, known as Retrieval-Augmented Generation (RAG), has become a cornerstone for building factually grounded and context-aware AI systems. This section reviews the evolution of RAG methodologies and reviews the key techniques that enable advanced retrieval from diverse data structures, including unstructured text, relational databases, and knowledge graphs.

A. Pre-Retrieval Processing

The quality of retrieval first begins with the initial user query as the amount of information and what to retrieve are highly dependent on how much context and how clear the user query is. To address this, query expansion has emerged as an essential technique to enhance retrieval by bridging vocabulary mismatch and enriching the user query's semantics to guide the retriever into finding more relevant candidates [6].

One such way of query expansion is Hypothetical Document Embeddings (HyDE) which generates a hypothetical answer to a query and embedding that document to search for semantically similar factual documents. This approach captures the semantic intent of the query more effectively compared to the initial user query that lacks context, leading to improved recall and precision [7].

B. Post-Retrieval Processing

After retrieval is conducted and a set of initial documents is retrieved, these documents typically are noisy but semantically similar to the first-stage retrieval as it is optimised for speed and recall [8]. Hence, these documents need to be refined to provide the LLM with the most relevant and accurate context for generating an accurate response.

To refine the initial documents set, a re-ranker is employed to compute the relevance score of each retrieved passage and

output the most relevant results, top-k results (e.g k=5). The re-ranker does this by concatenating the user's query and document's text into a single sequence separated by a special token. The combined sequence is then fed into a transformer model and as it passes through the transformer's layers, the self-attention mechanism allows every token from the query to interact with every token from the document to enable the model to capture complex, fine-grained relationships, contextual nuances and keyword importance that are missed when computing the cosine similarity between the query and document in the initial document retrieval [9].

C. Retrieval from Various Sources

Real-world data are rarely homogenous, often compromising a rich variety of formats that extend far beyond simple text such as images, embedded tables, charts, CSV files and other structured formats [10]. As each of these formats requires specialised handling, extending RAG beyond unstructured data to query structured or graph datastores, requires specialised Natural Language to query translation to accurately translate user intent into appropriate query language for each datastore.

For structure datastores or relational databases, Text-to-SQL methods have been an active area of research. As it enables users to gain insights into relational databases while overcoming the skill gap of authoring correct SQL queries [11]. This task has evolved significantly from early rule-based systems to advanced deep learning models and currently, LLMs.

With the use of LLMs for Text-to-SQL tasks, prompt engineering becomes the main focus as effective prompting is essential to providing the LLM with all the necessary context to understand the database's structure and user's intent [12]. To improve accuracy of SQL queries generated by the LLM, query-time table retrieval and query-time row retrieval is introduced. Query-time table retrieval aims to select and rank tables that are relevant to the user's query after the initial retrieval of candidates [13]. Query-time row retrieval complements this by providing the LLM with the table's schema but also specific relevant rows to aid the LLM in formulating precise SQL queries with correct values and avoid hallucination of filter conditions [14].

Similarly, for graph datastores that model complex relationships, the challenge lies in translating natural language into cypher queries. Equivalent to Text-to-SQL, the text-to-cypher task aims to bridge the skill gap of writing correct cypher queries themselves. This is done by providing the LLM with the user's question along with the graph schema, which dictates the nodes, relationships and properties with the database [15].

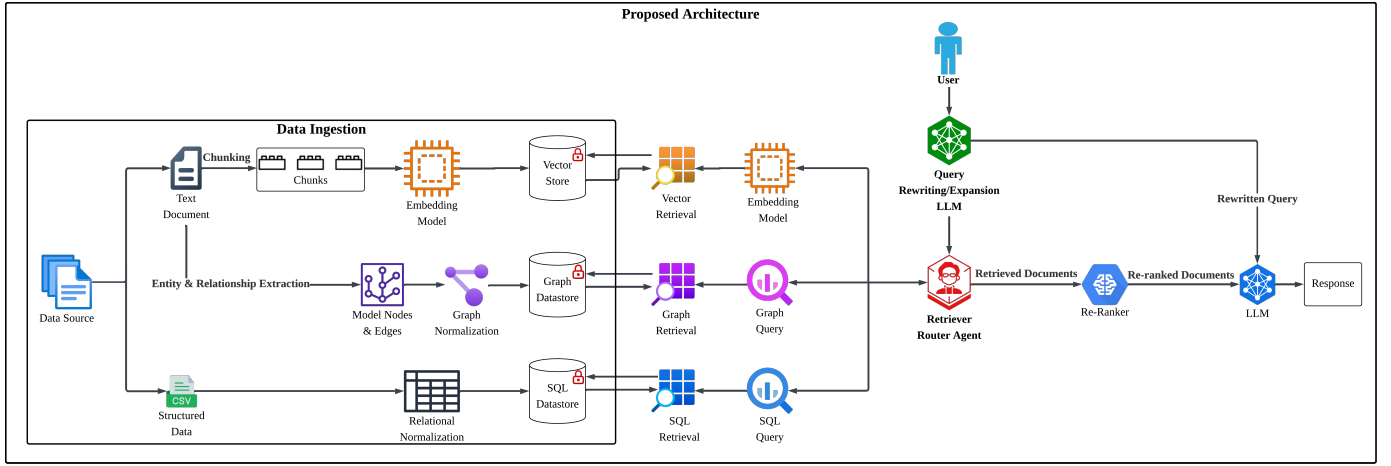


Fig. 1: AMS-RAG System Architecture

IV. AMS-RAG SYSTEM ARCHITECTURE

This paper implements and evaluates the foundational work for the Agentic Multi-Store RAG (AMS-RAG) architecture proposed in the previous work. The main focus of this paper is on the implementation and optimisation of the three core data pipelines: vector, SQL and graph. By establishing the most optimal configurations for each of these pipelines, it will overcome the limitations of a single-store RAG system by integrating into a complete agentic system as seen in (Figure 1).

V. EXPERIMENTAL METHODOLOGY

To evaluate the performance of the specialised RAG pipelines, a rigorous experimental setup comprising of tailored datasets, comprehensive evaluation framework and defined baseline configurations is established. This methodology ensures that the results are reproducible and that the impact of each optimisation technique is measured accurately.

A. Datasets and Benchmarking

The evaluation is grounded in a benchmark dataset of 100 question and answer pairs that was synthetically generated. Google’s Gemini 2.5 Pro was used to create this dataset and was prompted to act as a domain expert and formulate questions based on the content of the source documents. For each question, it also generated the exact context passage used to formulate the answer.

These datasets were intentionally composed of different formats such as unstructured text in PDFs and structured data in CSV files, to reflect the operational reality that different Home Team Departments (HDTs) store and manage their data in various formats.

This generation process was performed separately for each data modality:

- **Vector Dataset:** The knowledge source for this pipeline consists of the Singapore Police Force (SPF) Annual Crime Reports. These PDF documents contain unstructured text, tables and statistics related to crime trends,

making them ideal for evaluating text-based semantic retrieval

- **SQL Dataset:** This pipeline was tested against a structured database derived from Singapore Prison Service (SPS) on statistics of inmates in the form of CSV files. The data was ingested into a relational database and questions were designed to require precise lookups, joins and aggregations that can only be answered with SQL queries.
- **Graph Dataset:** The knowledge source for the graph pipeline is the Home Team Science and Technology Agency (HTX) Annual Report. A knowledge graph was constructed by extracting key entities such as ‘Projects’, ‘Mission’, ‘Departments’ and their relationships from the PDF. The questions for this pipeline were formulated to test the multi-hop reasoning and the discovery of complex connections within the data.

B. Evaluation Metrics

Ragas evaluation framework was employed to quantitatively assess the performance of each pipeline. The following key metrics scored on a scale of 0 to 1, were used for the vector pipeline:

- **Answer Relevancy:** Measures how well the generated answer addresses the user’s query.
- **Faithfulness:** Evaluates the factual consistency of the answer against the retrieved context. A low faithfulness score indicates hallucinations.
- **Context Recall:** Assesses whether all necessary ground truth context was retrieved by the pipeline.
- **Context Precision:** Measures the signal-to-noise ration of the retrieved context, where a high score indicates that the context is highly relevant and concise.

For the SQL pipeline, to evaluate the correctness of SQL queries generated by the LLM, two additional metrics were introduced:

- **DataCompy Score:** Direct comparison of the data returned by the generated query against the data from the

ground truth query.

- **Semantic Equivalence:** Measures whether the generated query is semantically identical to the ground truth query even if the syntax differs slightly.

As there is no native Ragas support for cypher queries, a custom metric, CypherMatch, was developed to evaluate cypher queries. This metric is derived from the DataCompy Score metric as it evaluates the accuracy of the generated cypher query by executing both it and the ground truth cypher query against the knowledge graph. CypherMatch uses the Jaccard similarity formula to compare the results of the executed queries.

The Jaccard similarity index is a statistic used to measure the similarity and diversity of sample sets. It is calculated by dividing the size of intersection of two sets by the size of their union. The formula is expressed as:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (1)$$

In the context of CypherMatch, 'A' and 'B' represent the results returned by the generated cypher query and ground truth cypher query respectively.

- **Numerator** $|A \cap B|$: Count of all the identical results that appear in both the generated query's output and the ground truth query's output.
- **Denominator** $|A \cup B|$: Count of all the unique results from both queries combined which eliminates duplicates.

The formula produces a score between 0 and 1, with 1 being a perfect match when all retrieved results are common and 0 indicating that the two queries returned completely different results.

C. Base Pipeline Definition

To measure the impact of each optimisation technique, a "Base Pipeline" for each datastore was established. These baselines represent a naive or simple RAG implementation and serve as the starting point for comparisons.

- **Base Vector Pipeline:** This pipeline uses Nanonets OCR to load parse PDF documents from the source. The documents are then segmented into text chunks using SentenceSplitter with a chunk size of 1024 characters and a chunk overlap of 200 characters. For the embedding model, intfloat/multilingual-e5-large was used. The resulting nodes are indexed in a VectorStoreIndex and retrieval is performed by querying this index, which returns the top-k most similar nodes based on vector similarity search.
- **Base SQL Pipeline:** This pipeline first ingests structured data from CSV files into an in-memory SQLite database using SQLiteDatabase from LlamaIndex and the sqlalchemy library. To generate SQL queries from natural language, it employs a zero-shot prompting strategy. The prompt provides the LLM with the complete database schema and the user's question but does not include any few-shot examples or advanced contextual information about

the tables. NLSQLTableQueryEngine is used as the query engine.

- **Base Graph Pipeline:** This pipeline constructs a knowledge graph within a Neo4j instance, managed via the Neo4jGraphStore. The graph's structure, including nodes like 'Organization', 'Person', 'Project', and relationships such as 'HASGROUPING' and 'CONTAINSCATEGORY', is populated through a manual ingestion process where triplets are explicitly defined and loaded. For query generation, the system uses a zero-shot prompt that provides the LLM with the graph schema and instructs it to generate a single, syntactically correct Cypher query. This baseline does not use any automated entity extraction modules.

VI. METHODS & IMPLEMENTATION

The three pipelines - Vector, SQL and Graph - were implemented as described in Section IV with each baseline serving as the starting point for iterative optimisation experiments. This section presents the benchmarking methodology and quantitative evaluation of the base pipelines with description and results of the subsequent improvements that led to the advanced versions.

Metric	SDR	Nanonets OCR
Answer Relevancy	0.76	0.67
Faithfulness	0.65	0.58
Context Recall	0.68	0.61
Context Precision	0.63	0.58

TABLE I: Performance comparison between document parsers SimpleDirectoryReader (SDR) and Nanonets OCR

A. Vector Pipeline

1) Base Vector Pipeline

In the initial phase of the vector pipeline development, two document parsing methods were evaluated to determine their suitability for ingesting and chunking unstructured text from PDFs that contain texts and visualisations. The first method uses SimpleDirectoryReader (SDR) loader from LlamaIndex, which based on the file extension, selects the best file reader and in the case of PDF files, it uses an internal PDFReader class that relies on standard Python libraries like pypdf. pypdf is then used to open the PDF file and extract embedded text directly and splits it into chunks for indexing.

The second approach uses Nanonets OCR system to extract text from scanned PDF image to ensure that the layout, structure and visual elements are preserved for analysis. It then conducts the following tasks simultaneously:

- **Text Recognition:** Identifying and transcribing characters from the page image.
- **Layout Analysis:** Understand the document's structure by identifying distinct elements such as paragraphs, headings, lists and tables.
- **Element Classification:** Recognise semantic elements such as LaTeX equations, signatures and watermarks to tag them.

Both methods were benchmarked on retrieval and question answering tasks in Table I using Ragas metrics such as answer relevancy, faithfulness, context recall and context precision.

The results revealed a trade-off between the two parsers. Although SDR achieved better overall performance across all Ragas metrics, this was primarily due to its direct and clean text extraction which preserved the raw integrity of the document’s content and resulted in higher scores for text-based questions.

However, for a subset of questions that required referencing data from visualisations such as tables, charts or graphs, the Nanonets OCR parser demonstrated a distinct advantage in its ability to process and tag semantic elements within the document structure, allowing it to extract information from non-textual sources effectively. To illustrate this, consider the following example based on a chart in the source document:

- **Question:** *”Based on the chart in the report, In 2022, which scam type resulted in \$97.6 million in losses?”*
- **Nanonets OCR Pipeline Answer:** *Government Officials Impersonation Scam.*
- **SDR Pipeline Answer:** *Investment scams.*

Annex A

Top 10 scam types in Singapore (Based on number of reported cases)

Types of Scams	Cases Reported		Total Amount Reported to Have Been Cheated (at least)	
	2022	2021	2022	2021
Phishing Scam	7,097	5,023	\$16.5M	\$34.8M
Job Scam	6,492	4,550	\$117.4M	\$91.0M
E-Commerce Scam	4,762	2,729	\$21.3M	\$5.9M
Investment Scam	3,108	2,467	\$198.3M	\$190.2M
Fake Friend Call Scam	2,106	686	\$8.8M	\$4.5M
Social Media Impersonation Scam	1,696	1,614	\$3.7M	\$5.5M
Loan Scam	1,031	2,273	\$9.3M	\$18.3M
Internet Love Scam	868	1,094	\$35.7M	\$46.6M
Government Officials Impersonation Scam	771	750	\$97.6M	\$106.3M
Credit-For-Sex Scam	626	628	\$2.1M	\$1.3M
Top 10 scams	28,557	21,814	\$511.3M	\$504.8M

Note: Total amount reported to have been cheated may not tally due to rounding.

Fig. 2: The source document chart used for the example question. The Nanonets OCR pipeline correctly identified ”Government Officials Impersonation Scam” as the cause of \$97.6 million in losses, while the SDR pipeline failed to extract this information from the visual element.

When processed using the Nanonets OCR, the LLM was able to generate the correct answer for the question. In contrast, the SDR was unable to interpret the visual element and answered with the incorrect answer.

This example highlights the capability of Nanonets OCR to extract information from visualisations that is not shown in the overall metric scores due to most of the questions being text-based. The slightly lower overall scores were due to the semantic tags (<table>, </figure>) attached to the texts which was a result of the markdown formatting used when parsing the PDFs. This increases the semantic distance between the vector representations of the generated answers and the questions. Hence, during retrieval, despite containing key information, these chunks are not identified as relevant due to them not resembling a natural sentence.

Given the importance of both text and visual data in the source documents, this capability was prioritised and Nanonets OCR was selected for the final advanced vector pipeline.

Metric	Nanonets OCR	Nanonets OCR + HyDE	Nanonets OCR + Re-ranker	Nanonets OCR + HyDE + Re-ranker + BM25
Answer Relevancy	0.67	0.68	0.70	0.72
Faithfulness	0.58	0.67	0.72	0.73
Context Recall	0.61	0.74	0.73	0.81
Context Precision	0.58	0.63	0.70	0.74

TABLE II: Overall Performance Comparison

2) Benchmarking Pre-retrieval Technique

To evaluate the efficiency of pre-retrieval enhancement, the performance of the base pipeline is benchmarked against the pipeline augmented with Hypothetical Document Embeddings (HyDE). As detailed in Table II, the integration of HyDE yielded improvements across all aspects of the Ragas metrics proving that HyDE is an effective pre-retrieval technique that enhances the RAG pipeline.

3) Benchmarking Post-Retrieval Techniques

To quantify the impact of post-retrieval processing, the performance of base pipelines is benchmarked against the pipeline augmented with a cross-encoder re-ranker. As shown in Table II, the re-ranker obtained significant performance gains over the base pipeline in all Ragas metrics. The most obvious improvements were observed in the faithfulness metric where there was a substantial increase from 0.58 to 0.72, indicating that the re-ranker is filtering out noisy or irrelevant context.

This is further supported by the results from the context precision and context recall where it is confirmed that the re-ranked context provided more relevant context than noise. These improvements indicate that the post-retrieval re-ranker is an essential component in optimising the RAG pipeline as it enhances the precision and reliability of the generated response.

4) Advanced Vector Pipeline

In the final stage of experimentation, the advanced vector pipeline was the result of synthesising the techniques into a single pipeline. The configuration of the advanced vector pipeline was build upon the pre-retrieval enhancement of

HyDE and the post-retrieval refinement of the cross-encoder re-ranker. Additionally, the semantic retrieval was augmented with the keyword-based BM25 retriever to tackle specialised terms or keywords. The results from both these retrievers were fused and passed to the re-ranker to ensure accuracy in the retrieval process.

The performance of this advanced vector pipeline, as shown in Table II, produced the highest scores across all evaluated metrics. The most significant improvement lies in the context recall metric where it peaked at 0.81, highlighting the significance of the addition of the BM25 retriever component that retrieves documents containing the exact keywords that the semantic search might miss.

With a more relevant set of initial candidates from retrieval, the re-ranker is able to perform its task more effectively, leading to higher scores in context precision and faithfulness. This indicates that the combination of these tools provided the most factually consistent information to the LLM for answer generation.

Overall, the advanced vector pipeline which integrates pre-retrieval and post-retrieval techniques and hybrid retrieval, stands as the best configuration for vector retrieval.

B. SQL Pipeline

1) Base SQL Pipeline

The base SQL pipeline was established to create a performance benchmark for the creation of the advanced SQL pipeline. It consists of a dynamic data ingestion process that identifies the CSV files within the directory and ingests them into an in-memory SQLite database. The Text-to-SQL generation process then takes place by using the NLSQLTable-QueryEngine, a library by LlamaIndex that translate natural language questions into executable SQL queries, to retrieve data from the datastore. The retrieved data is then used in the synthesis step where the LLM generates a response that is conversational based on the retrieved data.

Metric	Base	Base + QTTR	Base + QTRR	Arctic-Text2SQL	Base + QTTR + QTRR
Answer Relevancy	0.72	0.93	0.61	0.91	0.92
DataCompyScore	0.96	0.99	0.93	0.99	0.98
Semantic Equivalence	0.59	0.46	0.11	0.46	0.45

TABLE III: Overall Performance Comparison

2) Base SQL Pipeline Performance

Evaluating the performance of the SQL pipeline cannot be fully addressed by the standard Ragas metrics like faithfulness, context precision and context recall as these metrics primarily assess the quality of the final natural language response in relation to the retrieved context. In a SQL pipeline, the most crucial step is the generation of the SQL query to conduct retrieval as a incorrect or a non-executable query would lead to an empty retrieval and produce an unhelpful answer.

To overcome this limitation and to provide a more accurate evaluation, two SQL specialised metrics were used to evaluate Text-to-SQL tasks:

- **DataCompyScore:** An execution-based metric that measures the accuracy of the data retrieved by using the DataCompy library to perform a row-by-row comparison of results from both the generate SQL query and the ground truth SQL query. It outputs a F1-measure of precision and recall to indicate whether the generated SQL query was able to successfully retrieved the correct data.
- **Semantic Equivalence:** A non-execution metric that compares whether the generated SQL query is semantically identical to the ground truth SQL query even when the syntax is different.

By employing these two SQL-specific metrics, the results will provide a more rigorous and meaningful measure of the pipeline’s competency.

3) Benchmarking Query-Time Retrieval Techniques

The initial analysis of the base SQL pipeline as shown in Table III, achieved a high DataCompyScore of 0.96 which indicates the generated SQL queries were mostly retrieving the correct data. While the high data accuracy was promising, the Semantic Equivalence score was low which posed a risk for consistency. To address this and further improve performance, two context-aware retrieval techniques were implemented:

- **Query-Time Table Retrieval (QTTR):** This technique is similar to a re-ranker where instead of showing the entire database schema to the LLM to conduct retrieval, it selects only the tables that are most relevant to the user’s query to be shown to the LLM. This reduces the complexity and helps the LLM to only work with relevant data for the construction of the SQL query.
- **Query-Time Row Retrieval (QTRR):** This technique builds upon the QTTR by taking a few sample rows from the tables and providing it to the LLM as examples of data within each column. This results in the LLM having a better semantic understanding of the table’s content and knowing the exact queries to use to retrieve the table’s content.

As shown in Table III, the addition of QTTR has improved the answer relevancy score significantly and pushed the DataCompyScore to a near perfect which proves that by directing the model’s attention to relevant tables, it is able to produce better results. However, there was a decrease in semantic equivalence which shows the SQL queries generated by the LLM is semantically different from the ground truth SQL queries despite being able to retrieve the same data.

Conversely, adding QTRR proved to be detrimental instead as the scores for all metrics have seen a decrease from the base indicating the providing samples rows introduced noise that confused the LLM, leading to poor query generation.

Based on these results, QTTR has been identified as a valuable optimisation technique for improving the accuracy and answer relevance of the pipeline which supports its case to be included in the finalised SQL pipeline.

4) Advanced SQL Pipeline

In addition to experimenting with techniques to improve the overall performance of the pipeline, another LLM was

also used to conduct performance comparison. One such LLM is the Snowflake/Arctic-Text2SQL which is specifically fine-tuned for Text-to-SQL tasks making it a suitable candidate for generating highly accurate SQL queries from the user query.

In this LLM comparison, the advance pipeline uses the Llama-3.1-8B-Instruct model with QTTT and QTRR with the Arctic-Text2SQL having the same configurations as well besides the differing LLM. The Arctic-Text2SQL model excelled in DataCompyscore (0.99) and performed well on Answer Relevancy (0.91) while the Llama-3-based model had relatively good results with Answer Relevancy (0.92) and DataCompyscore (0.98) as well.

The performance of both advanced configurations showed an improvement over the base pipeline but still trail in comparison to the *Base + QTTT*. Hence, the finalised pipeline will be using the Llama-3.1-8B-Instruct model with QTTT technique.

C. Graph Pipeline

The final component of the experiment is the graph pipeline, which utilises a knowledge graph to handle complex queries that involves multi-hop relationships within the data. The knowledge graphs in this experiment was constructed manually based on domain expertise due to the lack of automated Text-to-Graph solutions.

1) Base Graph Pipeline

The base graph pipeline was established to create a performance benchmark for the creation of the advanced Graph pipeline. The LLM is provided with the user’s questions to generate cypher queries. To ensure the validity of the cypher queries, a custom class that is responsible for cleaning and validating cypher queries was implemented.

Metric	Base	microsoft/GraphRAG	Base + ED
Answer Relevancy	0.19	<u>0.57</u>	0.36
Cypher Match	0.06	-	<u>0.25</u>

TABLE IV: Graph Performance Comparison

2) Base Graph Pipeline performance

Similar to the SQL pipeline, standard Ragas metrics are insufficient for evaluating the correctness of Text-to-Cypher generation. To address this, a custom execution-based metric for cypher query, that is adapted from DataCompyscore is used to assess the accuracy of the generated cypher query. This is done by running both the generated cypher query and the ground truth query against the knowledge graph and comparing the dataframes of the returned results.

As shown in Table IV, the naive approach to the graph pipeline performed poorly with a low answer relevancy score of 0.17 and a cypher match score of only 0.06. A manual analysis revealed that the main point of failure is due to the model’s inability to generate accurate cypher queries which resulted in retrieving blanks as outputs.

3) Microsoft’s GraphRAG

To provide a robust benchmark for the graph pipeline, Microsoft’s GraphRAG, a highly regarded solution for question answering over knowledge graphs was evaluated. Unlike the

graph pipeline in this experiment which focuses on generating explicit Text-to-Cypher queries, GraphRAG operates by identifying relevant communities of nodes within the graph based on the user’s query before synthesising a summary from the textual content of those communities.

As seen from Table IV, GraphRAG achieved the highest answer relevancy score of 0.57 which indicates it is effective in producing contextually appropriate answers. As mentioned in the previous paragraph that GraphRAG does not generate cypher queries for retrieval but instead relies on community detection, it cannot be evaluated using the CypherMatch metric as there is no generated cypher query to compare against the ground truth.

Instead, GraphRAG performs a semantic search against a vector index of all the text chunk nodes to find the most semantically similar chunk to the user’s query which will be the initial chunk. The algorithm will then traverse the graph to find a “community” of closely related nodes and uses this list of most relevant text chunk nodes to be used as context for answer generation.

4) Advanced Graph Pipeline

The *Base + ED* (Entity Disambiguation) pipeline builds upon the base configuration by addressing the lack of context awareness in the Text-to-Cypher generation process. This pipeline implementation introduces providing the graph schema and incorporating entity disambiguation to better equip the LLM in generating accurate cypher queries.

The graph schema details all available node labels, relationships and properties to the LLM so that it does not need to guess the architecture. Entity disambiguation, maps ambiguous terms like acronyms to their specific entries in the knowledge graph before the LLM is shown the graph schema. This is achieved by building an entity map during the construction of the knowledge graph and using the APOC library from Neo4j to merge all synonym nodes into a single node. The APOC library rewires all incoming and outgoing relationships from the synonym nodes to a single node before deleting the now-redundant duplicates. For example,

User query: “Who is the **chief executive officer** of HTX?”

Disambiguated input to LLM: “Who is the node **CEO** of the entity **HTX**?”

As shown in Table IV, the impact of these enhancements is significant, where the cypher match score increases from 0.06 in the base pipeline to 0.25 in the *Base + ED* configuration. This improvement highlights that by providing the LLM with context on the graph schema and disambiguation, enables it to generate accurate cypher queries.

Overall, while the GraphRAG produce the highest score in answer relevance, this metric proved to be misleading as a deeper analysis revealed that the model often produced relevant sounding answers even when no relevant context was retrieved. For example,

User query: "What is the Open Innovation Challenge?"

Generated Answer: "The Open Innovation Challenge is typically a competition or initiative designed to encourage individuals, teams, or organizations to develop innovative solutions to specific problems or challenges. These challenges often invite participants from various backgrounds to collaborate and contribute ideas, leveraging open innovation principles to drive creativity and problem-solving. The specific details and objectives of an Open Innovation Challenge can vary depending on the organization or entity hosting it."

Ground Truth Answer: "It's Hatch's accelerator-style call where startups submit solutions to real Home Team problem statements; the best are selected to refine and test their tech in operational settings."

This generated response is a hallucination as it provides a generic definition of an "Open Innovation Challenge" but contains none of the specific details from the ground truth. This behaviour inflates the answer relevancy score as the text is relevant to the topic but factually ungrounded and incorrect within the datastore.

The tendency to generate answers without relevant context makes it unsuitable for the AMS-RAG where accuracy is prioritised. Unlike GraphRAG's summarisation approach, the advanced graph pipeline ensures every answer is grounded in retrieved facts from the knowledge graph. Additionally, as the cypher query for retrieval is visible, it enables traceability at times of failure instead of generating ungrounded information. Hence, the advanced graph pipeline would be the viable choice in moving towards the final integrated system.

VII. CHALLENGES & LIMITATIONS

The implementation of techniques and tools to optimise the pipelines were successful but also subjected to several practical and technical challenges.

A key constraint was the project requirement to be fully local which prevents the use of cloud-based APIs such as Nanonets API and LlamaParse. Initially, these two APIs were explored for their document parsing capabilities which generated excellent results especially the Nanonets API.

A challenge posed by the vector pipeline was the custom hybrid retriever class as LlamaIndex does not offer a pre-built ensemble retriever class. This hybrid retriever was used to combine the BM25 retriever and vector search to be executed in parallel and re-rank the results from both of these retrieval methods.

The graph pipeline presented many significant hurdles as it is a field that is relatively new and there are few research done in this area. Although current generation of LLMs are proficient at many tasks, they still struggle with accurately converting unstructured text into a knowledge graph. In this paper, LLMs such as Llama 3.1, gpt-4o and gpt-5 were used to experiment with automatic generation of knowledge

graph from a PDF file. Gpt-5 was the best performing out of the three, but even so, the entities and relationships in the knowledge graph were mostly inaccurate.

As a result, the knowledge graph was created manually to avoid this single component becoming a bottleneck and halting the overall implementation of the AMS-RAG system. It was necessary trade-off to enable benchmarking of the base graph pipeline and also to evaluate the optimisation techniques that can be applied to the graph pipeline.

Finally, a notable limitation was encountered in the evaluation of the graph pipeline where the Ragas framework which has been used for evaluation in the vector and SQL pipelines do not have metrics that assess the correctness of Cypher queries. To overcome this hurdle, the SQL metrics, DataCompyScore, was adapted to evaluate Cypher queries instead of SQL queries. This allowed a more accurate and consistent assessment of the graph pipeline's performance.

VIII. CONCLUSION & FUTURE WORK

This paper has presented a comprehensive evaluation of the specialised RAG pipelines for vector, SQL and graph datastores. Through benchmarking, the optimised configurations for each datastores has been identified, proving that a one-size-fits-all RAG approach is not optimal in an environment with multiple data types.

For the final synthesis step, all pipelines will standardize on the Llama-3.1-8B-Instruct model as the generation component. Based on the findings, the final configurations for each pipeline are as follows:

- **Vector Pipeline:** Consists of a local Nanonets OCR parser to handle text and visual data, a hybrid retriever that executes both semantic search and BM25 keyword search in parallel and a cross-encoder re-ranker for post-retrieval.
- **SQL Pipeline:** Consists of meta-llama/Llama-3.1-8B-Instruct model for Text-to-SQL tasks and leveraging on QTTR for retrieving relevant tables.
- **Graph Pipeline:** Consists of the meta-llama/Llama-3.1-8B-Instruct model for Text-to-Cypher generation, leveraging on graph schema and entity disambiguation module to ensure the generation of accurate and verifiable queries.

The successful optimisation of the individual pipelines in this paper lay the groundwork for the next phase of this research which will be as follows:

- **Experimentation with Agents:** The first step will be to familiarise and find out the process of building an agent and what can it do.
- **Integration of Pipelines:** Integrate the current three pipelines into a unified multi-store architecture that is able to retrieve data from unstructured, structured and graph-based datastores simultaneously.
- **Development of Orchestrator Agent:** Building an orchestrator agent to manage the integrated pipeline in which it will intelligently determine the nature of the user's request and routing it to the most appropriate

datastore. Essentially, the orchestrator agent will be the central 'brain' of the AMS-RAG system that ensures the most efficient and effective retrieval path is always chosen.

REFERENCES

- [1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Apr. 2021. Available: <https://arxiv.org/pdf/2005.11401>
- [2] Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," Mar. 2024. Available: <https://arxiv.org/pdf/2312.10997>
- [3] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models Chain-of-Thought Prompting," Jan. 2023. Available: <https://arxiv.org/pdf/2201.11903>
- [4] L. Gao, X. Ma, J. Lin, and J. Callan, "Precise Zero-Shot Dense Retrieval without Relevance Labels," Dec. 2022. Available: <https://arxiv.org/pdf/2212.10496>
- [5] A. Singh, A. Ehtesham, S. Kumar, and Khoei, Tala Talaei, "Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG," arXiv (Cornell University), Jan. 2025, doi: <https://doi.org/10.48550/arxiv.2501.09136>.
- [6] Y. Xia et al., "Knowledge-Aware Query Expansion with Large Language Models for Textual and Relational Retrieval," arXiv (Cornell University), Oct. 2024, doi: <https://doi.org/10.48550/arxiv.2410.13765>.
- [7] W. Fan et al., "A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models." Available: <https://arxiv.org/pdf/2405>.
- [8] G. Izacard and E. Grave, "Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering," arXiv.org, Feb. 03, 2021. <https://arxiv.org/abs/2007.01282v2>
- [9] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," Jan. 2019, doi: <https://doi.org/10.48550/arxiv.1901.04085>.
- [10] Nazarii Drushchak et al., "Multimodal Retrieval-Augmented Generation: Unified Information Processing Across Text, Image, Table, and Video Modalities," pp. 59–64, Jan. 2025, doi: <https://doi.org/10.18653/v1/2025.magmar-1.5>.
- [11] A. Kumar, P. Nagarkar, P. Nalhe, and S. Vijayakumar, "Deep Learning Driven Natural Languages Text to SQL Query Conversion: A Survey," arXiv.org, 2022. <https://arxiv.org/abs/2208.04415v1>.
- [12] D. Gao et al., "Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation," Proceedings of the VLDB Endowment, vol. 17, no. 5, pp. 1132–1145, Jan. 2024, doi: <https://doi.org/10.14778/3641204.3641221>.
- [13] D. Li, K. Bi, J. Guo, and X. Cheng, "Bridging Queries and Tables through Entities in Table Retrieval," arXiv.org, 2025. <https://arxiv.org/abs/2504.06551>.
- [14] Z. Wu et al., "RB-SQL: A Retrieval-based LLM Framework for Text-to-SQL," arXiv (Cornell University), Jul. 2024, doi: <https://doi.org/10.48550/arxiv.2407.08273>.
- [15] Ozsoy, Makbule Gulcin, L. Messallem, J. Besga, and G. Minneci, "Text2Cypher: Bridging Natural Language and Graph Databases," arXiv (Cornell University), Dec. 2024, doi: <https://doi.org/10.48550/arxiv.2412.10064>.